

Day1

CSP-J 模拟赛

时间：2026 年 7 月 13 日 07:45 ~ 11:15

题目名称	数位跃迁	秘符石板	果树引枝	小樱的库洛牌
题目类型	传统型	传统型	传统型	传统型
可执行文件名	jump	slate	tree	clow
输入文件名	jump.in	slate.in	tree.in	clow.in
输出文件名	jump.out	slate.out	tree.out	clow.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	10	20	20	10
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	jump.cpp	slate.cpp	tree.cpp	clow.cpp
-----------	----------	-----------	----------	----------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项与提醒（请选手务必仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 `main()` 的返回值类型必须是 `int`，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参照具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 程序可使用的栈内存空间限制与题目的内存限制一致。
7. 只提供 Linux 格式附加样例文件。
8. 评测在 NOI Linux 下进行，各语言的编译器版本以其为准。

数位跃迁 (jump)

【题目描述】

在数位宇宙中，存在着由 n 个二进制位构成的固定跃迁空间。这个空间里的每一个节点都对应一个非负整数，且所有节点的数值都不会超过 $2^n - 1$ ——这意味着每个节点都能被完整映射为 n 位二进制数，高位空缺时自动补 0，确保跃迁过程中的数位对齐。

你的任务是启动数位跃迁程序，将初始节点 s 精准跃迁到目标节点 t 。每次跃迁都需要遵循严格的空間规则，且会消耗一定的能量，我们的核心目标是找到能耗总和最低的跃迁路径。单次跃迁的执行规则如下：选择一个目标节点 x 作为本次跃迁的落点，跃迁生效的核心条件是，当前所在节点与目标节点 x 进行**按位或**运算后，结果必须是 n 位全 1 的二进制数。跃迁完成后，你将从当前节点移动到 x ，本次跃迁的能耗等于跃迁前节点与 x 进行**按位异或**运算的结果。

跃迁次数没有任何限制，你可以连续进行多次合法跃迁，每次跃迁的起点都是上一次的落点。值得放心的是，无论 s 和 t 是空间中的哪个节点，都必然存在合法的跃迁路径。

现有 T 组独立的数位跃迁任务，每组任务给出三个整数 n, s, t ，其中 s 和 t 均为跃迁空间内的合法节点（即 $0 \leq s, t < 2^n$ ）。请为每组任务计算出：从初始节点 s 跃迁到目标节点 t 所需的最小能耗总和。

【输入格式】

从文件 *jump.in* 中读入数据。

第一行输入一个正整数 T ，代表跃迁任务的总组数。

接下来 T 行，每行依次输入三个整数 n, s, t ，分别表示跃迁空间的二进制位数、初始节点、目标节点。

【输出格式】

输出到文件 *jump.out* 中。

对于每组任务，单独输出一行一个整数，代表完成该跃迁任务的最小能耗总和。

【样例输入 1】

```
2
2 1 2
3 1 1
```

【样例输出 1】

3
0

【样例解释 1】

第一组任务中，跃迁空间的二进制位数为 2， n 位全 1 的数是 3。初始节点 1 与目标节点 2 进行按位或运算的结果恰好是 3，满足单次跃迁的生效条件。因此可以直接从 1 跃迁到 2，本次跃迁的能耗是 1 和 2 进行按位异或运算的结果 3，这也是该任务的最小能耗。

第二组任务中，初始节点与目标节点完全一致，无需执行任何跃迁操作，自然不需要消耗能量，能耗总和为 0。

【样例 2】

见选手目录下的 *ex_jump2.in* 和 *ex_jump2.out*。

该样例满足测试点 3 的性质。

【样例 3】

见选手目录下的 *ex_jump3.in* 和 *ex_jump3.out*。

该样例满足测试点 5 ~ 6 的性质。

【样例 4】

见选手目录下的 *ex_jump4.in* 和 *ex_jump4.out*。

该样例满足测试点 8 ~ 10 的性质。

【数据范围】

测试点编号	n	特殊性质
1 ~ 2	≤ 10	无
3	$= 1$	无
4	≤ 30	$t = 0$
5 ~ 6	≤ 30	$s t = 2^n - 1$
7	≤ 30	$t = 2^n - 1$
8 ~ 10	≤ 30	无

对于所有数据： $1 \leq T \leq 50, 1 \leq n \leq 30, 0 \leq s, t \leq 2^n$ 。

秘符石板 (slate)

【题目描述】

尘封的地宫深处，静卧着一方刻满秘符的古老石板。石板以星象分划，共设 n 层 m 格，每一格都镌有一枚源自上古的小写字母秘符。

地宫典籍记载，石板的序途始于顶层左角的「圣坛原点」 $(1, 1)$ ，终于底层右角的「核心祭台」 (n, m) 。探寻者只能沿石板的脉络移动：每一步仅可沿水平脉络右行一格，或沿垂直脉络下潜一格，绝不可逾越石板边界。

沿序途所经秘符，会依顺序凝成一道「符语」。两个符语相比，字典序更小的更优，你作为地宫的探寻者，需寻得这道最优符语。

【输入格式】

从文件 *slate.in* 中读入数据。

输入的第一行，包含两个正整数 N 和 M ，分别表示石板的层数与格数。

接下来 N 行，每行包含一个长度为 M 的字符串。字符串由小写英文字母构成，代表对应层的秘符排布。

【输出格式】

输出到文件 *slate.out* 中。

输出一行一个字符串，代表你寻得的最优符语。

【样例输入 1】

```
3 3
abc
def
ghi
```

【样例输出 1】

```
abcfi
```

【样例输入 2】

```
3 3
bbb
```

bba

bbb

【样例输出 2】

bbbab

【样例 3】

见选手目录下的 *ex_slate3.in* 和 *ex_slate3.out*。

【数据范围】

测试点编号	n, m	特殊性质
1 ~ 2	$= 1$	无
3 ~ 4	≤ 10	无
5 ~ 8	≤ 200	A
9 ~ 12	≤ 1000	B
13 ~ 20	≤ 2000	无

特殊性质 A ：石板中任意一格的右侧与下侧格位，所刻秘符均不相同；

特殊性质 B ： $n \leq 2$ ；

对于所有数据： $1 \leq n, m \leq 2000$ 。

果树引枝 (tree)

【题目描述】

乡下的果园里, T 棵果树都长成了规整的树状模样。树上的每个节点都是一颗饱满的果实, 编号从 1 到 n , 1 号果实长在树干最顶端, 是整棵树的“根果”。除了根果外, 每颗果实都只通过一根细枝, 连接着另一颗编号更小的果实, 枝条交错却从不缠绕。

为了让果实长得更匀称, 果农需要用透气的绑带固定所有细枝。绑带的使用有几个贴心的小要求:

1. 每一根细枝都必须被一条绑带覆盖, 不能重复包裹, 也不能漏掉任何一根, 不然会影响枝条的养分输送;
2. 绑带的起点必须选在“末梢果实”上, 也就是有根树的叶子节点。
3. 绑带可以准备任意多条, 长短也能自由裁剪, 绑带的长度按覆盖的细枝数量计算, 但太长的绑带容易被风吹落, 所以果农想找到一种绑缚方案, 让所有绑带里最长的那一条, 长度尽可能短。

请为每棵果树找到满足要求的绑缚方案, 算出方案中最长绑带的最小可能长度, 然后输出这个数值。

【输入格式】

从文件 *tree.in* 中读入数据。

第一行输入一个整数 T , 代表果树的数量。

对于每棵果树:

第一行输入整数 n , 代表果实的总数;

第二行输入 $n - 1$ 个整数, 依次是 p_2, p_3 直到 p_n , 每个 p_i 的取值在 1 到 $i - 1$ 之间, 分别对应 2 号到 n 号果实的父果编号。

【输出格式】

输出到文件 *tree.out* 中。

共 T 行, 每行一个整数, 表示第 i 棵果树的答案。

【样例输入 1】

```
2
8
```

```
1 2 3 2 5 1 7
8
1 2 3 4 5 6 7
```

【样例输出 1】

```
3
7
```

【样例解释 1】

第一棵果树的枝条分支错落，最优绑缚方案用了三条绑带，从 4,6,8 三个末梢果实从下往上绑带长度分别是 2,3,2，最长的那条是 3。

第二棵果树的枝条是笔直的链式，只有 8 号果实是末梢果实。要覆盖所有 7 根细枝，绑带只能从 8 号果实一直系到 1 号果实，长度就是 7，没有其他方案。

【样例 2】

见选手目录下的 *ex_tree2.in* 和 *ex_tree2.out*。

该数据点满足测试点 5 ~ 8 的性质。

【样例 3】

见选手目录下的 *ex_tree3.in* 和 *ex_tree3.out*。

该数据点满足测试点 9 ~ 20 的性质。

【数据范围】

测试点编号	n	特殊性质
1 ~ 4	≤ 5	无
5 ~ 8	≤ 20	T=1
9 ~ 20	$\leq 2 \times 10^5$	无

对于所有数据： $1 \leq T \leq 10^5, 2 \leq n \leq 2 \times 10^5, 1 \leq p_i < i, \sum n \leq 2 \times 10^5$ 。

小樱的库洛牌 (clow)

【题目描述】

木之本樱掌管着 n 张编号为 1 到 n 的库洛牌，这些牌原本按特殊方式排列——没有任何一张库洛牌出现在与自己编号相同的位置。这是库洛里多定下的禁忌规则，违反会触发魔法紊乱。

调皮的斯比奈鲁·太阳偷偷闯入房间，打乱了部分库洛牌的摆放。它拿走了一些位置的牌，只留下空白标记，其余位置的库洛牌则保持原样。小樱想知道，有多少种方式能将空白位置补上库洛牌，使补全后的排列仍满足库洛里多的禁忌规则，且所有库洛牌恰好出现一次形成完整排列。

斯比虽调皮却未破坏原排列的合法性，当前非空白位置的库洛牌都不在对应禁忌位置，且每张非空白库洛牌的编号都是唯一的。空白位置至少有两个，且一定存在至少一种合法补全方式。

请帮助小樱计算合法补全方案数，答案对 1000000007 取模。

【输入格式】

第一行包含一个整数 n ，表示库洛牌的数量。

第二行包含 n 个整数，表示被斯比打乱后的库洛牌排列，其中 -1 表示空白位置，其余数表示对应位置上保留的库洛牌编号。

【输出格式】

输出一个整数，表示合法的补全方案数，答案对 1000000007 取模。

【样例输入 1】

```
5
-1 -1 4 3 -1
```

【样例输出 1】

```
2
```

【样例解释 1】

合法的补全方式有两种：

1. 位置 1 放 2，位置 2 放 5，位置 5 放 1，排列为 $[2, 5, 4, 3, 1]$ ；

2. 位置 1 放 5，位置 2 放 1，位置 5 放 2，排列为 [5,1,4,3,2]。

两种方式均满足所有库洛牌不在对应禁忌位置。

【样例 2】

见选手目录下的 *ex_clow2.in* 和 *ex_clow2.out*。

【数据范围】

测试点编号	n	特殊性质
1 ~ 2	≤ 10	无
3 ~ 4	≤ 100	A
5 ~ 6	≤ 2000	无
7	$\leq 10^5$	A
8 ~ 10	$\leq 10^5$	无

特殊性质 A：每个空白位置 i 对应的编号 i 的库洛牌未出现在非空白位置中；
对于所有数据： $2 \leq n \leq 10^5$ 。